

1.DATA IN R

Source Code :

```
# Create a Data Frame
```

```
students <- data.frame(  
  Name = c("Alice", "Bob", "Charlie", "David"),  
  Age  = c(22, 25, 23, 24),  
  Score = c(85, 90, 78, 88)  
)
```

```
# Print the data frame
```

```
print(students)
```

```
# Access a specific column
```

```
print(students$Name)
```

```
# Filter data (students with Score > 80)
```

```
high_scores <- students[students$Score > 80, ]  
print(high_scores)
```

```
# Add a new column
```

```
students$Passed <- students$Score > 80  
print(students)
```

```
# Compute the mean score
```

```
mean_score <- mean(students$Score)  
print(paste("Average Score:", mean_score))
```

Output :

Name Age Score

1 Alice 22 85

2 Bob 25 90

3 Charlie 23 78

4 David 24 88

[1] "Alice" "Bob" "Charlie" "David"

Name Age Score

1 Alice 22 85

2 Bob 25 90

4 David 24 88

Name Age Score Passed

1 Alice 22 85 TRUE

2 Bob 25 90 TRUE

3 Charlie 23 78 FALSE

4 David 24 88 TRUE

[1] "Average Score: 85.25"

2.READING AND WRITING DATA

Source code :

Create a Data Frame

```
students <- data.frame (Name = c("Alice", "Bob", "Charlie", "David"),Age  
= c(22, 25, 23, 24),
```

```
Score = c(85, 90, 78, 88)
```

```
)
```

```
# Write data to a CSV file
```

```
write.csv (students, "students.csv", row.names = FALSE)
```

```
print("Data successfully written to students.csv")
```

```
#Read the data back from the CSV file
```

```
students_data <- read.csv("students.csv")
```

```
# Print the data read from the file
```

```
print ("Data read from students.csv:")
```

```
print (students_data)
```

Output :

```
[1] "Data successfully written to students.csv"
```

```
[1] "Data read from students.csv:"
```

	Name	Age	Score
1	Alice	22	85
2	Bob	25	90
3	Charlie	23	78
4	David	24	88

3.R AND DATABASE

Source code :

```
# Install packages (Run once if not installed)
# install.packages("DBI")
# install.packages("RMySQL")

# Load required libraries
library(DBI)
library(RMySQL)

# Establish connection to MySQL database
con <- dbConnect(
  RMySQL::MySQL(),
  dbname  = "test_db",  # Avoid spaces in DB name
  host    = "localhost",
  user    = "root",
  password = "1234"
)

# Create a table if it does not exist
query_create <- "
CREATE TABLE IF NOT EXISTS students (
  ID INT AUTO_INCREMENT PRIMARY KEY,
  Name VARCHAR(50),
  Age INT,
```

```

    Score FLOAT
);
"

dbExecute(con, query_create)

# Insert sample data into the table
query_insert <- "
INSERT INTO students (Name, Age, Score) VALUES
('Alice', 22, 85),
('Bob', 25, 90),
('Charlie', 23, 78),
('David', 24, 88);
"

dbExecute(con, query_insert)

# Retrieve data from the database
query_select <- "SELECT * FROM students;"

students_data <- dbGetQuery(con, query_select)

# Print retrieved data
print("Data retrieved from database:")
print(students_data)

# Close the connection
dbDisconnect(con)

```

Output :

[1] "Data retrieved from database:"

	ID	Name	Age	Score
1	1	Alice	22	85
2	2	Bob	25	90
3	3	Charlie	23	78
4	4	David	24	88
5	5	Alice	22	85
6	6	Bob	25	90
7	7	Charlie	23	78
8	8	David	24	88

4.DATES

Source code :

```
# Load necessary package
library(lubridate)

# 1. Get the current date and time
current_date <- Sys.Date()
current_time <- Sys.time()
timezone <- Sys.timezone()

# 2. Convert a string to Date and POSIXct
datel <- as.Date("2025-04-03")
datetimeel <- as.POSIXct("2025-04-03 14:30:00")

# 3. Extract components of the date
year_val <- format(datel, "%Y")    # Year
month_val <- format(datel, "%m")   # Month
day_val <- format(datel, "%d")     # Day
weekday_val <- weekdays(datel)     # Weekday name
month_name <- months(datel)         # Month name
quarter_val <- quarters(datel)     # Quarter

# 4. Date arithmetic
date_after_7_days <- datel + 7
date_difference <- as.Date("2025-04-10") - datel
```


5. Create a sequence of dates

```
date_seq <- seq.Date(from = as.Date("2025-04-01"),  
                     to = as.Date("2025-04-10"), by = "days")
```

6. First and last day of the month

```
first_day <- floor_date(datel, "month")  
last_day <- ceiling_date(datel, "month") - 1
```

7. Print outputs

```
cat("Current Date:", current_date, "\n")  
cat("Current Time:", current_time, "\n")  
cat("Timezone:", timezone, "\n\n")  
  
cat("Converted Date:", datel, "\n")  
cat("Converted Date-Time:", datetimetel, "\n\n")  
  
cat("Year:", year_val, "\n")  
cat("Month:", month_val, "\n")  
cat("Day:", day_val, "\n")  
cat("Weekday:", weekday_val, "\n")  
cat("Month Name:", month_name, "\n")  
cat("Quarter:", quarter_val, "\n\n")  
  
cat("Date After 7 Days:", date_after_7_days, "\n")  
cat("Date Difference in Days:", date_difference, "\n\n")  
cat("Sequence of Dates:", date_seq, "\n\n")
```

```
cat("First Day of Month:", first_day, "\n")
```

```
cat("Last Day of Month:", last_day, "\n")
```

Output :

Current Date: 2025 -04-03

Current Time: 2025 -04-03 14:35:12 UTC

Timezone: Asia/Calcutta

Converted Date: 2025 -04-03

Converted Date-Time: 2025 -04-03 14:35:12 UTC

Year: 2025

Month: 04

Day: 03

Weekday: Thursday

Month Name: April

Quarter: Q2

Date After 7 Days: 2025 -04-03

Date Difference in Days: Time different of 7 days

Sequence of Dates: 2025-04-01 2025-04-02 2025-04-03 2025-04-04
2025-04-05 2025-04-06 2025-04-07 2025-04-08 2025-04-10

First Day of Month: 2025-04-01

Last Day of Month: 2025-04-30

5.FACTORS

Source code :

```
# Creating a Factor for Categorical Data
colors <- factor (c("Red", "Blue", "Green", "Blue", "Red", "Green", "Red"))
# Display the factor
print ("Factor Levels and Values:")
print (colors)
print ("Levels of the factor:")
print (levels (colors))
#Count occurrences of each level
print ("Count of Each Level:")
print (table (colors))
# Changing the order of levels
colors <- factor (colors, levels = c("Green", "Blue", "Red"))
print ("Reordered Factor Levels:")
print (colors)
# Adding a new level
colors <- factor (colors, levels = c(levels (colors),"Yellow"))
print ("Factor after adding new level:")
print (levels (colors))
# Checking the frequency of each category
print ("Frequency table of colors:")
print (summary (colors))
#Convert Factor to Character
color_char <- as.character (colors)
print("Converted Factor to Character:")
```

```
print (color_char)
# Convert Factor to Numeric (based on level ordering)
color_num <- as.numeric (colors)
print("Converted Factor to Numeric:")
print (color_num)
```

Output :

```
[1] Red  Blue  Green Blue  Red  Green Red
```

```
Levels: Blue Green Red
```

```
[1] "Levels of the factor:"
```

```
[1] "Blue" "Green" "Red"
```

```
[1] "Count of Each Level:"
```

```
colors
```

```
Blue Green  Red
```

```
2  2  3
```

```
[1] "Reordered Factor Levels:"
```

```
[1] Red  Blue  Green Blue  Red  Green Red
```

```
Levels: Green Blue Red
```

```
[1] "Factor after adding new level:"
```

```
[1] "Green" "Blue" "Red"  "Yellow"
```

```
[1] "Frequency table of colors:"
```

```
Green  Blue  Red Yellow
```

```
2  2  3  0
```

```
[1] "Converted Factor to Character:"
```

```
[1] "Red" "Blue" "Green" "Blue" "Red" "Green" "Red"
```

```
[1] "Converted Factor to Numeric:"
```

```
[1] 3 2 1 2 3 1 3
```


6.SUBSCRIBING

Source code :

```
# Create a sample data frame
data <- data.frame(
  ID = 1:5,
  Name = c("Alice", "Bob", "Charlie", "David", "Eve"),
  Age = c(25, 30, 35, 40, 28),
  Gender = factor(c("Female", "Male", "Male", "Male", "Female")),
  Score = c(88, 76, 90, 85, 92)
)

# Display the entire data frame
cat("Original Data Frame:\n")
print(data)

# 1. Subsetting a single column
cat("\nExtracting 'Name' column:\n")
print(data$Name)

# 2. Subsetting multiple columns
cat("\nExtracting 'Name' and 'Age' columns:\n")
print(data[, c("Name", "Age")])

# 3. Subsetting specific rows based on condition (Age > 30)
cat("\nRows where Age > 30:\n")
print(subset(data, Age > 30))
```

4. Subsetting using row and column indices

```
cat("\nElement at Row 3, Column 2 (Name of 3rd Person):\n")
```

```
print(data[3, 2])
```

5. Subsetting first 3 rows

```
cat("\nFirst 3 Rows:\n")
```

```
print(data[1:3, ])
```

6. Subsetting using logical conditions (Extracting Male entries)

```
cat("\nSubset where Gender is Male:\n")
```

```
print(subset(data, Gender == "Male"))
```

7. Subsetting and selecting specific columns

```
cat("\nMale entries with Name and Score only:\n")
```

```
print(subset(data, Gender == "Male", select = c("Name", "Score")))
```

Output :

Original Data Frame:

	ID	Name	Age	Gender	Score
1	1	Alice	25	Female	88
2	2	Bob	30	Male	76
3	3	Charlie	35	Male	90
4	4	David	40	Male	85
5	5	Eve	28	Female	92

Extracting 'Name' column:

```
[1] "Alice" "Bob"   "Charlie" "David" "Eve"
```

Extracting 'Name' and 'Age' columns:

	Name	Age
1	Alice	25
2	Bob	30
3	Charlie	35
4	David	40
5	Eve	28

Rows where Age > 30:

	ID	Name	Age	Gender	Score
3	3	Charlie	35	Male	90
4	4	David	40	Male	85

Element at Row 3, Column 2 (Name of 3rd Person):

[1] "Charlie"

First 3 Rows:

	ID	Name	Age	Gender	Score
1	1	Alice	25	Female	88
2	2	Bob	30	Male	76
3	3	Charlie	35	Male	90

Subset where Gender is Male:

	ID	Name	Age	Gender	Score
2	2	Bob	30	Male	76
3	3	Charlie	35	Male	90
4	4	David	40	Male	85

Male entries with Name and Score only:

	Name	Score
2	Bob	76
3	Charlie	90
4	David	85

7.CHARACTER MANIPULATION

Source code :

```
# Load stringr package
```

```
library(stringr)
```

```
# 1. Create a Character Vector
```

```
names <- c("Alice", "Bob", "Charlie", "David", "Eve")
```

```
# 2. Convert to Uppercase and Lowercase
```

```
cat("Uppercase Names:\n")
```

```
print(toupper(names))
```

```
cat("\nLowercase Names:\n")
```

```
print(tolower(names))
```

```
# 3. Get String Length
```

```
cat("\nLength of Each Name:\n")
```

```
print(nchar(names))
```

```
# 4. Concatenation of Strings
```

```
full_names <- paste(names, "Smith", sep = "")
```

```
cat("\nFull Names with Concatenation:\n")
```

```
print(full_names)
```

```
# 5. Substring Extraction
```

```
cat("\nExtracting First 3 Characters of Each Name:\n")
```

```
print(substr(names, 1, 3))
```

6. String Replacement

```
modified_names <- sub("e", "3", names) # Replace first occurrence of 'e'  
with '3'
```

```
cat("\nReplacing 'e' with '3' in Names:\n")
```

```
print(modified_names)
```

7. Splitting a sentence into words

```
sentence <- "R programming is fun"
```

```
split_words <- str_split(sentence, pattern = " ")[[1]]
```

```
cat("\nSplitting Sentence into Words:\n")
```

```
print(split_words)
```

8. String Padding (Adding Leading Zeros)

```
numbers <- c("5", "23", "100")
```

```
padded_numbers <- str_pad(numbers, width = 5, side = "left", pad = "0")
```

```
cat("\nPadded Numbers with Leading Zeros:\n")
```

```
print(padded_numbers)
```

9. Detecting a Pattern in Strings

```
has_A <- grepl("A", names, ignore.case = TRUE)
```

```
cat("\nNames Containing 'A':\n")
```

```
print(has_A)
```

10. Counting Specific Characters in a String

```
num_e <- str_count(names, "e")  
cat("\nNumber of 'e' in Each Name:\n")  
print(num_e)
```

Output :

Uppercase Names:

```
[1] "ALICE" "BOB" "CHARLIE" "DAVID" "EVE"
```

Lowercase Names:

```
[1] "alice" "bob" "charlie" "david" "eve"
```

Length of Each Name:

```
[1] 5 3 7 5 3
```

Full Names with Concatenation:

```
[1] "AliceSmith" "BobSmith" "CharlieSmith" "DavidSmith"
"EveSmith"
```

Extracting First 3 Characters of Each Name:

```
[1] "Ali" "Bob" "Cha" "Dav" "Eve"
```

Replacing 'e' with '3' in Names:

```
[1] "Alic3" "Bob" "Charli3" "David" "Ev3"
```

Splitting Sentence into Words:

```
[1] "R" "programming" "is" "fun"
```

Padded Numbers with Leading Zeros:

```
[1] "00005" "00023" "00100"
```

Names Containing 'A':

```
[1] TRUE FALSE TRUE TRUE FALSE
```

Number of 'e' in Each Name:

```
[1] 1 0 1 0 1
```


8.DATA AGGREGATION

Source code :

```
# Load necessary package
```

```
library(dplyr)
```

```
# Create a sample data frame
```

```
data <- data.frame(
```

```
  Department = c("HR", "IT", "HR", "Finance", "IT", "Finance", "HR", "IT"),
```

```
  Employee = c("Alice", "Bob", "Charlie", "David", "Eve", "Frank", "Grace",  
"Hank"),
```

```
  Salary = c(50000, 60000, 55000, 70000, 65000, 72000, 53000, 67000),
```

```
  Experience = c(3, 5, 4, 8, 6, 9, 2, 7)
```

```
)
```

```
# Display the original data
```

```
cat("Original Data:\n")
```

```
print(data)
```

```
# 1. Aggregate Total Salary by Department
```

```
salary_aggregate <- aggregate(Salary ~ Department, data, sum)
```

```
cat("\nTotal Salary by Department:\n")
```

```
print(salary_aggregate)
```

```
# 2. Aggregate Average Salary by Department
```

```
avg_salary <- aggregate(Salary ~ Department, data, mean)
```

```
cat("\nAverage Salary by Department:\n")
```

```
print(avg_salary)
```

3. Aggregate Count of Employees by Department

```
emp_count <- aggregate(Employee ~ Department, data, length)
cat("\nNumber of Employees in Each Department:\n")
print(emp_count)
```

4. Aggregate Minimum and Maximum Salary by Department

```
min_salary <- aggregate(Salary ~ Department, data, min)
max_salary <- aggregate(Salary ~ Department, data, max)
cat("\nMinimum Salary by Department:\n")
print(min_salary)
cat("\nMaximum Salary by Department:\n")
print(max_salary)
```

5. Using dplyr for Aggregation

```
agg_data <- data %>%
  group_by(Department) %>%
  summarise(
    Total_Salary = sum(Salary),
    Max_Salary = max(Salary),
    Min_Salary = min(Salary),
    Avg_Salary = mean(Salary),
    Employee_Count = n(),
    .groups = "drop"
  )
cat("\nAggregated Data using dplyr:\n")
print(agg_data)
```

Output :

Original Data:

	Department	Employee	Salary	Experience
1	HR	Alice	50000	3
2	IT	Bob	60000	5
3	HR	Charlie	55000	4
4	Finance	David	70000	8
5	IT	Eve	65000	6
6	Finance	Frank	72000	9
7	HR	Grace	53000	2
8	IT	Hank	67000	7

Total Salary by Department:

	Department	Salary
1	Finance	142000
2	HR	158000
3	IT	192000

Average Salary by Department:

	Department	Salary
1	Finance	71000.00
2	HR	52666.67
3	IT	64000.00

Number of Employees in Each Department:

	Department	Employee
1	Finance	2
2	HR	3
3	IT	3

Minimum Salary by Department:

	Department	Salary
1	Finance	70000
2	HR	50000
3	IT	60000

Maximum Salary by Department:

	Department	Salary
1	Finance	72000
2	HR	55000
3	IT	67000

Aggregated Data using dplyr:

A tibble: 3 × 6

	Department	Total_Salary	Max_Salary	Min_Salary	Avg_Salary	Employee_Count
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<int>
1	Finance	<u>142000</u>	<u>72000</u>	<u>70000</u>	<u>71000</u>	2
2	HR	<u>158000</u>	<u>55000</u>	<u>50000</u>	<u>52667</u>	3
3	IT	<u>192000</u>	<u>67000</u>	<u>60000</u>	<u>64000</u>	3

9.RESHAPING DATA BASICS

Source code :

```
# Install packages if not already installed
if (!require(tidyr)) install.packages("tidyr")
if (!require(reshape2)) install.packages("reshape2")

# Load libraries
library(tidyr)
library(reshape2)

# Create a sample data frame (Wide Format)
data <- data.frame(
  ID = c(1, 2, 3),
  Name = c("Alice", "Bob", "Charlie"),
  Math = c(85, 78, 92),
  Science = c(90, 88, 85),
  English = c(88, 76, 80)
)

# Display the original wide-format data
print("Original Wide Data:")
print(data)

# 1. Convert Wide Data to Long Format using pivot_longer
long_data <- pivot_longer(
  data,
```

```
cols = c(Math, Science, English),  
names_to = "Subject",  
values_to = "Score"  
)
```

```
print("Long Format Data:")  
print(long_data)
```

2. Convert Long Data back to Wide Format using pivot_wider

```
wide_data <- pivot_wider(  
  long_data,  
  names_from = Subject,  
  values_from = Score  
)
```

```
print("Converted Back to Wide Format:")  
print(wide_data)
```

3. Convert Wide to Long using melt (reshape2)

```
long_data_melt <- melt(  
  data,  
  id.vars = c("ID", "Name"),  
  variable.name = "Subject",  
  value.name = "Score"  
)
```

```
print("Long Format Using melt() from reshape2:")  
print(long_data_melt)
```

4. Convert Long to Wide using dcast

```
wide_data_cast <- dcast(  
  long_data_melt,  
  ID + Name ~ Subject,  
  value.var = "Score"  
)
```

```
print("Converted Back to Wide Using dcast():")  
print(wide_data_cast)
```

Output :

[1] "Original Wide Data:"

	ID	Name	Math	Science	English
1	1	Alice	85	90	88
2	2	Bob	78	88	76
3	3	Charlie	92	85	80

[1] "Long Format Data:"

A tibble: 9 × 4

	ID	Name	Subject	Score
	<dbl>	<chr>	<chr>	<dbl>
1	1	Alice	Math	85
2	1	Alice	Science	90
3	1	Alice	English	88
4	2	Bob	Math	78
5	2	Bob	Science	88
6	2	Bob	English	76
7	3	Charlie	Math	92
8	3	Charlie	Science	85
9	3	Charlie	English	80

[1] "Converted Back to Wide Format:"

A tibble: 3 × 5

	ID	Name	Math	Science	English
	<dbl>	<chr>	<dbl>	<dbl>	<dbl>
1	1	Alice	85	90	88
2	2	Bob	78	88	76


```
3 3 Charlie 92 85 80
```

```
[1] "Long Format Using melt() from reshape2:"
```

```
ID Name Subject Score
```

```
1 1 Alice Math 85
```

```
2 2 Bob Math 78
```

```
3 3 Charlie Math 92
```

```
4 1 Alice Science 90
```

```
5 2 Bob Science 88
```

```
6 3 Charlie Science 85
```

```
7 1 Alice English 88
```

```
8 2 Bob English 76
```

```
9 3 Charlie English 80
```

```
[1] "Converted Back to Wide Using dcast():"
```

```
ID Name Math Science English
```

```
1 1 Alice 85 90 88
```

```
2 2 Bob 78 88 76
```

```
3 3 Charlie 92 85 80
```

10.THE R ENVIRONMENT

Source code :

```
# Program to calculate Mean, Median and Mode from user input
# Get user input as a single line
numbers_input <- readline(prompt = "Enter numbers separated by
spaces: ")

# Convert input string into numeric vector (split by space)
numbers <- as.numeric(unlist(strsplit(numbers_input, " ")))

# Check for invalid input
if (any(is.na(numbers))) {
  stop("Please enter only numeric values separated by spaces.")
}

# Calculate Mean
mean_value <- mean(numbers)

# Calculate Median
median_value <- median(numbers)

# Calculate Mode
get_mode <- function(x) {
  uniq_vals <- unique(x)
  uniq_vals[which.max(tabulate(match(x, uniq_vals)))]
}
```

```
mode_value <- get_mode(numbers)
```

```
# Print outputs
```

```
cat("Mean:", mean_value, "\n")
```

```
cat("Median:", median_value, "\n")
```

Output :

Enter numbers separated by spaces: 88 25 52 95 11

Mean= 54.2

Median= 52

Mode= 88

11.PROBABILITY AND DISTRIBUTIONS

Source code:

```
# Set seed for reproducibility
```

```
set.seed(123)
```

```
# 1. Uniform Distribution
```

```
print("Random Numbers from Uniform Distribution (0 to 1):")
```

```
uniform_sample <- runif(5, min = 0, max = 1)
```

```
print(uniform_sample)
```

```
# 2. Normal Distribution (Mean = 0, SD = 1)
```

```
print("Random Numbers from Normal Distribution (Mean = 0, SD = 1):")
```

```
normal_sample <- rnorm(5, mean = 0, sd = 1)
```

```
print(normal_sample)
```

```
# 3. Binomial Distribution (n = 10, p = 0.5)
```

```
print("Random Numbers from Binomial Distribution (n = 10, p = 0.5):")
```

```
binomial_sample <- rbinom(5, size = 10, prob = 0.5)
```

```
print(binomial_sample)
```

```
# 4. Poisson Distribution (Lambda = 4)
```

```
print("Random Numbers from Poisson Distribution (Lambda = 4):")
```

```
poisson_sample <- rpois(5, lambda = 4)
```

```
print(poisson_sample)
```

```
# 5. Exponential Distribution (Rate = 1)
```

```
print("Random Numbers from Exponential Distribution (Rate = 1):")  
exp_sample <- rexp(5, rate = 1)  
print(exp_sample)
```

6. CDF and Quantile functions for Normal Distribution

```
print("Cumulative Probability  $P(X \leq 1)$  for Normal Distribution (Mean =  
0, SD = 1):")  
print(pnorm(1, mean = 0, sd = 1))
```

```
print("95th Percentile of Normal Distribution (Mean = 0, SD = 1):")  
print(qnorm(0.95, mean = 0, sd = 1))
```

7. Plotting Normal Distribution

```
x <- seq(-4, 4, length = 100)  
y <- dnorm(x, mean = 0, sd = 1)  
  
plot(x, y,  
      type = "l",  
      main = "Normal Distribution PDF",  
      xlab = "x",  
      ylab = "Density",  
      col = "blue",  
      lwd = 2)
```

Output :

[1] "Random Numbers from Uniform Distribution (0 to 1):"

[1] 0.2875775 0.7883051 0.4089769 0.8830174 0.9404673

[1] "Random Numbers from Normal Distribution (Mean = 0, SD = 1):"

[1] -1.6895557 1.2394959 -0.1089660 -0.1172420 0.1830826

[1] "Random Numbers from Binomial Distribution (n = 10, p = 0.5):"

[1] 7 4 2 4 8

[1] "Random Numbers from Poisson Distribution (Lambda = 4):"

[1] 7 5 5 10 5

[1] "Random Numbers from Exponential Distribution (Rate = 1):"

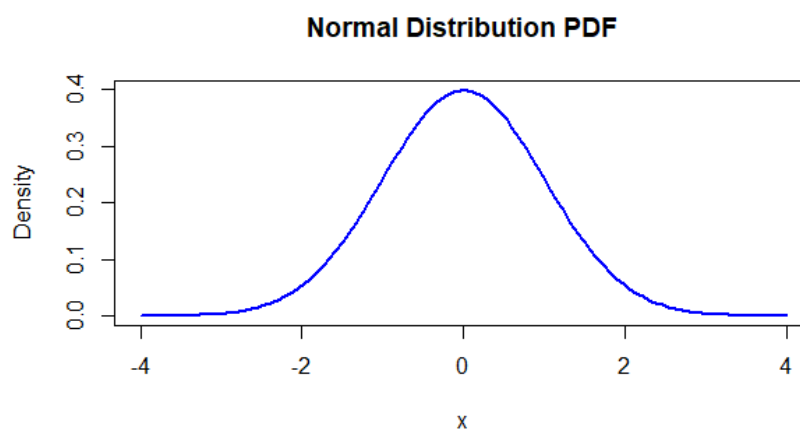
[1] 0.41706094 0.08813205 0.18828404 0.84978613 1.56320354

[1] "Cumulative Probability $P(X \leq 1)$ for Normal Distribution (Mean = 0, SD = 1):"

[1] 0.8413447

[1] "95th Percentile of Normal Distribution (Mean = 0, SD = 1):"

[1] 1.644854



12.DESRIPTIVE STATISTICS AND GRAPHICS

Source code :

```
# Load necessary package
if (!require(ggplot2)) install.packages("ggplot2")
library(ggplot2)

# Create a sample dataset
data <- data.frame(
  Score = c(78, 85, 92, 88, 76, 90, 95, 80, 82, 87)
)

# 1. Basic Descriptive Statistics
print("Summary of Data:")
print(summary(data$Score))

print("Mean of Score:")
print(mean(data$Score))

print("Median of Score:")
print(median(data$Score))

print("Standard Deviation of Score:")
print(sd(data$Score))

print("Variance of Score:")
print(var(data$Score))
```



```
print("Range of Score:")  
print(range(data$Score))
```

```
print("Quantiles of Score:")  
print(quantile(data$Score))
```

2. Frequency Table (Categorized into 5 bins)

```
print("Frequency Table:")  
print(table(cut(data$Score, breaks = 5)))
```

3. Data Visualization

Histogram

```
hist(data$Score,  
      main = "Histogram of Scores",  
      col = "blue",  
      xlab = "Scores",  
      ylab = "Frequency",  
      border = "black")
```

Boxplot

```
boxplot(data$Score,  
        main = "Boxplot of Scores",  
        col = "green",  
        horizontal = TRUE)
```

```
# Density Plot using ggplot2
ggplot(data, aes(x = Score)) +
  geom_density(fill = "lightblue", alpha = 0.5) +
  labs(title = "Density Plot of Scores",
        x = "Scores",
        y = "Density")
```

Output :

[1] "Summary of Data:"

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
------	---------	--------	------	---------	------

76.0	80.5	86.0	85.3	89.5	95.0
------	------	------	------	------	------

[1] "Mean of Score:"

[1] 85.3

[1] "Median of Score:"

[1] 86

[1] "Standard Deviation of Score:"

[1] 6.236986

[1] "Variance of Score:"

[1] 38.9

[1] "Range of Score:"

[1] 76 95

[1] "Quantiles of Score:"

0%	25%	50%	75%	100%
----	-----	-----	-----	------

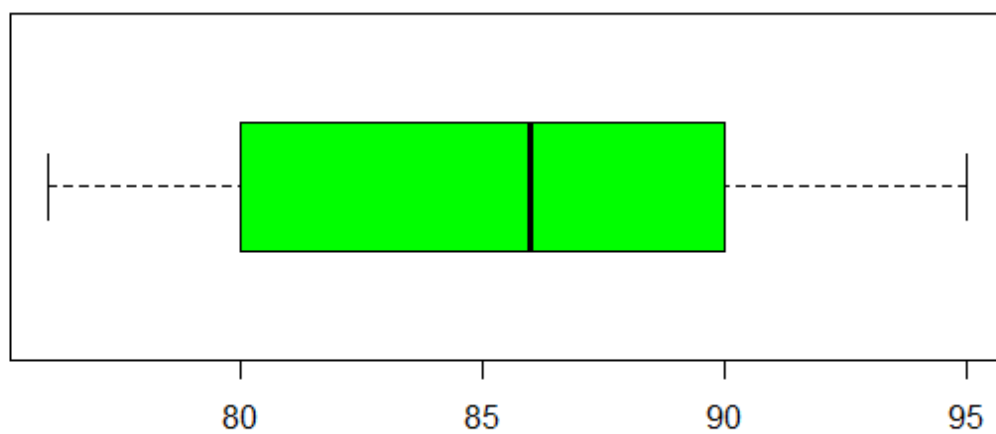
76.0	80.5	86.0	89.5	95.0
------	------	------	------	------

[1] "Frequency Table:"

(76,79.8]	(79.8,83.6]	(83.6,87.4]	(87.4,91.2]	(91.2,95]
-----------	-------------	-------------	-------------	-----------

2	2	2	2	
---	---	---	---	--

Boxplot of Scores



13.ONE - AND TWO - SAMPLE TESTS

Source code :

```
# Set seed for reproducibility
```

```
set.seed(123)
```

```
# Generate sample data
```

```
group1 <- rnorm(10, mean = 50, sd = 5) # Sample 1
```

```
group2 <- rnorm(10, mean = 55, sd = 5) # Sample 2
```

```
# 1. One-Sample t-Test
```

```
print("One-Sample t-Test (H0: Mean = 50)")
```

```
one_sample_t_test <- t.test(group1, mu = 50)
```

```
print(one_sample_t_test)
```

```
# 2. One-Sample Wilcoxon Test
```

```
print("One-Sample Wilcoxon Test (H0: Median = 50)")
```

```
one_sample_wilcox <- wilcox.test(group1, mu = 50)
```

```
print(one_sample_wilcox)
```

```
# 3. Two-Sample t-Test
```

```
print("Two-Sample t-Test (H0: Mean of Group1 = Mean of Group2)")
```

```
two_sample_t_test <- t.test(group1, group2, var.equal = TRUE)
```

```
print(two_sample_t_test)
```

4. Paired t-Test

```
paired_sample1 <- c(10, 20, 30, 40, 50)
```

```
paired_sample2 <- c(12, 19, 29, 39, 48)
```

```
print("Paired t-Test (H0: No difference in means)")
```

```
paired_t_test <- t.test(paired_sample1, paired_sample2, paired = TRUE)
```

```
print(paired_t_test)
```

5. Wilcoxon Rank-Sum Test (Mann-Whitney U Test)

```
print("Wilcoxon Rank-Sum Test (H0: Group1 = Group2)")
```

```
wilcox_rank_test <- wilcox.test(group1, group2)
```

```
print(wilcox_rank_test)
```

6. Chi-Square Test of Independence

```
observed <- matrix(c(30, 20, 50, 40), nrow = 2, byrow = TRUE)
```

```
print("Chi-Square Test of Independence")
```

```
chi_square_test <- chisq.test(observed)
```

```
print(chi_square_test)
```

Output :

[1] "One-Sample t-Test (H0: Mean = 50)"

One Sample t-test

data: group1

$t = 0.24742$, $df = 9$, $p\text{-value} = 0.8101$

alternative hypothesis: true mean is not equal to 50

95 percent confidence interval:

46.96165 53.78461

sample estimates:

mean of x

50.37313

[1] "One-Sample Wilcoxon Test (H0: Median = 50)"

Wilcoxon signed rank exact test

data: group1

$V = 27$, $p\text{-value} = 1$

alternative hypothesis: true location is not equal to 50

[1] "Two-Sample t-Test (H0: Mean of Group1 = Mean of Group2)"

Two Sample t-test

data: group1 and group2

$t = -2.5438$, $df = 18$, $p\text{-value} = 0.02036$

alternative hypothesis: true difference in means is not equal to 0

95 percent confidence interval:

-10.3528472 -0.98711591

sample estimates:

mean of x mean of y

50.37313 56.04311

[1] "Paired t-Test (H0: No difference in means)"Paired t-test

data: paired_sample1 and paired_sample2

$t = 0.88465$, $df = 4$, $p\text{-value} = 0.4263$

alternative hypothesis: true mean difference is not equal to 0

95 percent confidence interval:

-1.283077 2.483077

sample estimates:

mean difference

0.6

[1] "Wilcoxon Rank-Sum Test (H0: Group1 = Group2)"

Wilcoxon rank sum exact test

data: group1 and group2

$W = 23$, $p\text{-value} = 0.04326$

alternative hypothesis: true location shift is not equal to 0

[1] "Chi-Square Test of Independence"

Pearson's Chi-squared test with Yates' continuity correction

data: observed

$X\text{-squared} = 0.10954$, $df = 1$, $p\text{-value} = 0.7407$

14.REGRESSION AND CORRELATION

Source code :

```
# Load necessary package
if (!require(ggplot2)) install.packages("ggplot2")
library(ggplot2)

# Set seed for reproducibility
set.seed(123)

# Create sample dataset
data <- data.frame(
  X = rnorm(20, mean = 50, sd = 10)
)

# Create correlated Y variable
data$Y <- 2 * data$X + rnorm(20, mean = 0, sd = 5)

#### 1. Correlation Analysis ####
print("Correlation Coefficient between X and Y:")
correlation_value <- cor(data$X, data$Y)
print(correlation_value)

# Scatter plot with regression line
ggplot(data, aes(x = X, y = Y)) +
  geom_point(color = "blue") +
  geom_smooth(method = "lm", color = "red") +
```

```
labs(title = "Scatter Plot with Regression Line",  
      x = "X Variable",  
      y = "Y Variable")
```

```
### 2. Linear Regression Model ###
```

```
model <- lm(Y ~ X, data = data)
```

```
print("Regression Model Summary:")
```

```
print(summary(model))
```

```
# Predicted values
```

```
predictions <- predict(model,  
                        newdata = data.frame(X = c(45, 55, 65)))
```

```
print("Predicted values for X = 45, 55, 65:")
```

```
print(predictions)
```

```
# Plot Residuals
```

```
plot(model$residuals,  
      main = "Residuals Plot",  
      ylab = "Residuals",  
      xlab = "Index",  
      col = "red",  
      pch = 19)
```

```
abline(h = 0, col = "blue", lty = 2)
```

Output :

[1] "Correlation Coefficient between X and Y:"

[1] 0.9773254

[1] "Regression Model Summary:"

Call:

lm(formula = Y ~ X, data = data)

Residuals:

Min	1Q	Median	3Q	Max
-7.5615	-3.0093	-0.9287	4.1543	6.2955

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	1.7557	5.2353	0.335	0.741
X	1.9609	0.1001	19.582	1.39e-13 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

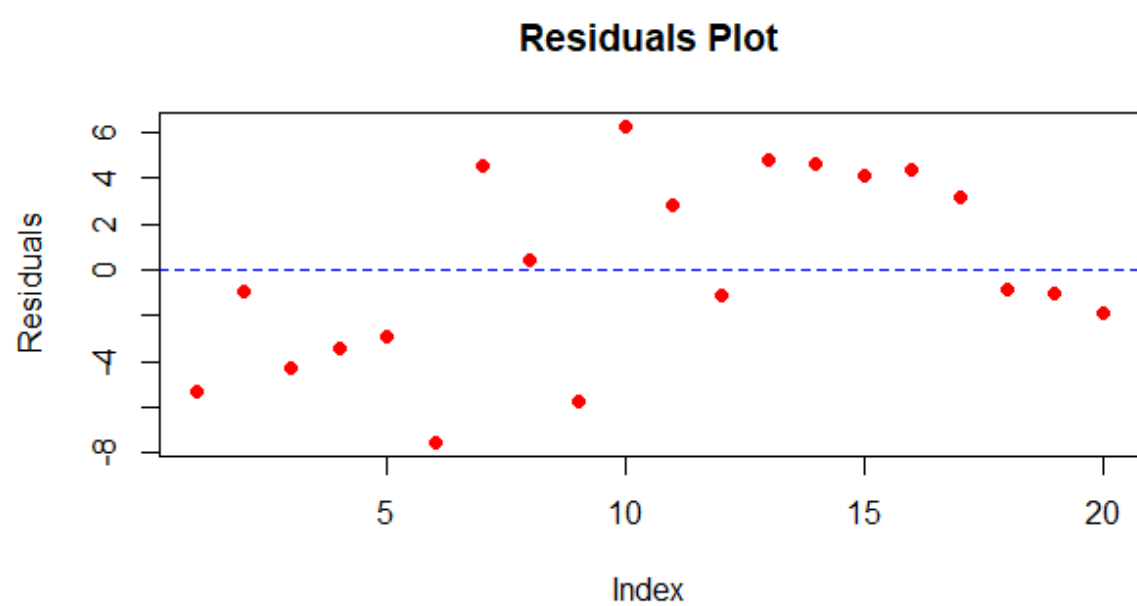
Residual standard error: 4.245 on 18 degrees of freedom

Multiple R-squared: 0.9552, Adjusted R-squared: 0.9527

F-statistic: 383.5 on 1 and 18 DF, p-value: 1.386e-13

[1] "Predicted values for X = 45, 55, 65:"

1	2	3
89.99479	109.60348	129.21216



15.ANALYSIS OF VARIANCE AND THE KRUSKAL - WALLIS TEST

Source code :

```
# ANOVA and Kruskal-Wallis Test

# Sample dataset: test scores for 3 groups
group <- factor(rep(c("A", "B", "C"), each = 5))

score <- c(85, 86, 88, 75, 78,
           82, 83, 85, 80, 84,
           90, 91, 92, 89, 88)

# Combine into a data frame
data <- data.frame(group, score)

print("Dataset =")
print(data)

# 1. One-Way ANOVA
anova_result <- aov(score ~ group, data = data)

print("ANOVA Result =")
print(summary(anova_result))

# 2. Kruskal-Wallis Test
```


Output :

[1] "Dataset ="

group score

1	A	85
2	A	86
3	A	88
4	A	75
5	A	78
6	B	82
7	B	83
8	B	85
9	B	80
10	B	84
11	C	90
12	C	91
13	C	92
14	C	89
15	C	88

[1] "ANOVA Result ="

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
group	2	182.9	91.47	7.317	0.00836 **
Residuals	12	150.0	12.50		

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

[1] "Kruskal-Wallis Test Result ="

Kruskal-Wallis rank sum test

data: score by group

Kruskal-Wallis chi-squared = 9.1376, df = 2, p-value = 0.01037

[1] "Post-hoc: Tukey HSD (ANOVA)"

Tukey multiple comparisons of means

95% family-wise confidence level

Fit: aov(formula = score ~ group, data = data)

\$group

	diff	lwr	upr	p adj
B-A	0.4	-5.565524	6.365524	0.9825390
C-A	7.6	1.634476	13.565524	0.0135479
C-B	7.2	1.234476	13.165524	0.0186848

[1] "Post-hoc: Pairwise Wilcoxon (Kruskal-Wallis)"

Pairwise comparisons using Wilcoxon rank sum test with continuity correction

data: score and group

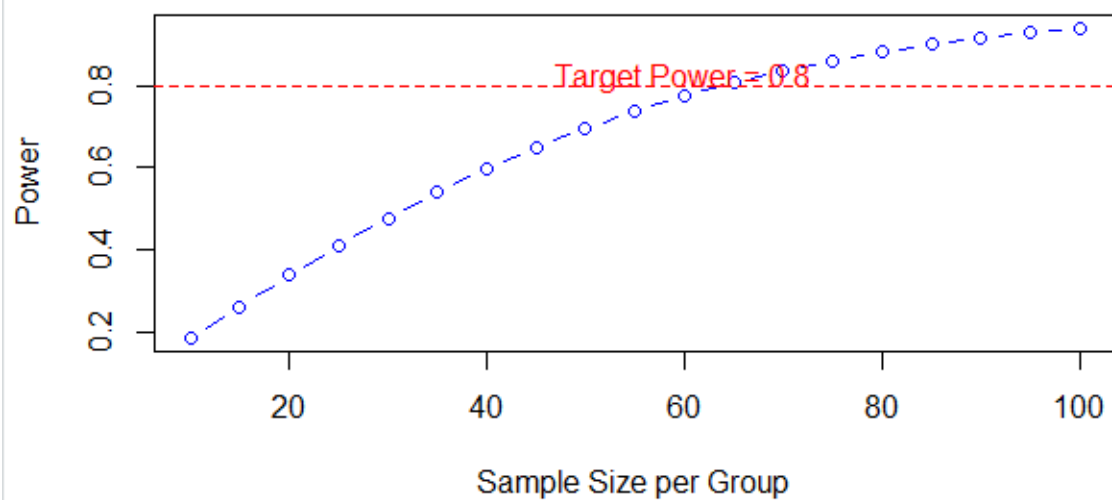
A B

B 1.000 -

C 0.048 0.024

P value adjustment method: bonferroni

Power vs Sample Size for $d = 0.5$



16.TABULAR DATA EXAMPLE

Source code :

```
# Create vectors for each column
name <- c("Alice", "Bob", "Charlie", "David", "Eva")
age <- c(20, 22, 23, 21, 20)
score <- c(85, 90, 88, 76, 95)

# Combine into a data frame (table)
students <- data.frame(
  Name = factor(name), # convert Name to factor to show in summary
  Age = age,
  Score = score
)

# Print the data frame
print("Student Data Table:")
print(students)

# Summary of the data (includes Name now)
print("Summary Statistics (including Name):")
print(summary(students))

# Access specific column
print("Student Scores:")
print(students$Score)
```

```
# Add a new column: Grade based on score
students$Grade <- ifelse(students$Score >= 90, "A",
                        ifelse(students$Score >= 80, "B", "C"))

# Convert Grade to factor for better summary
students$Grade <- factor(students$Grade, levels = c("A", "B", "C"))

print("Updated Table with Grades:")
print(students)

# Optional: Summary again to include Grade
print("Summary Statistics (including Name and Grade):")
print(summary(students))
```

Output :

[1] "Student Data Table:"

	Name	Age	Score
1	Alice	20	85
2	Bob	22	90
3	Charlie	23	88
4	David	21	76
5	Eva	20	95

[1] "Summary Statistics (including Name):"

	Name	Age	Score
Alice	:1	Min. :20.0	Min. :76.0
Bob	:1	1st Qu.:20.0	1st Qu.:85.0
Charlie	:1	Median :21.0	Median :88.0
David	:1	Mean :21.2	Mean :86.8
Eva	:1	3rd Qu.:22.0	3rd Qu.:90.0
		Max. :23.0	Max. :95.0

[1] "Student Scores:"

[1] 85 90 88 76 95

[1] "Updated Table with Grades:"

	Name	Age	Score	Grade
1	Alice	20	85	B
2	Bob	22	90	A
3	Charlie	23	88	B
4	David	21	76	C

5 Eva 20 95 A

[1] "Summary Statistics (including Name and Grade):"

Name	Age	Score	Grade
Alice	:1	Min. :20.0	Min. :76.0 A:2
Bob	:1	1st Qu.:20.0	1st Qu.:85.0 B:2
Charlie	:1	Median :21.0	Median :88.0 C:1
David	:1	Mean :21.2	Mean :86.8
Eva	:1	3rd Qu.:22.0	3rd Qu.:90.0
		Max. :23.0	Max. :95.0

17.POWER AND THE COMPUTATION OF SAMPLE SIZE

Source code :

```
# Power and Sample Size Computation in R
# Install pwr package if not already installed
if (!require(pwr)) install.packages("pwr", dependencies = TRUE)
library(pwr)

cat("POWER ANALYSIS AND SAMPLE SIZE CALCULATION\n\n")

# 1. Compute Sample Size Needed
# Medium effect size (d = 0.5), power = 0.8, alpha = 0.05
# For two-sample t-test (two-sided)
cat(">>> Calculating required sample size per group (medium effect
size, 80% power):\n")
sample_size <- pwr.t.test(d = 0.5, power = 0.8, sig.level = 0.05,
                          type = "two.sample", alternative = "two.sided")
print(sample_size)

# 2. Compute Power Given Sample Size
# Let's assume you only have 30 samples per group
cat("\n>>> Calculating power with only 30 samples per group:\n")
power_given_n <- pwr.t.test(n = 30, d = 0.5, sig.level = 0.05,
                           type = "two.sample", alternative = "two.sided")
print(power_given_n)
```

3. Optional: Plot Power vs Sample Size

```
cat("\n>>> Plotting Power vs. Sample Size (Effect size d = 0.5):\n")
sample_sizes <- seq(10, 100, by = 5)
powers <- sapply(sample_sizes, function(n) {
  pwr.t.test(n = n, d = 0.5, sig.level = 0.05, type = "two.sample")$power
})
```

Plotting

```
plot(sample_sizes, powers, type = "b", col = "blue",
      xlab = "Sample Size per Group", ylab = "Power",
      main = "Power vs Sample Size for d = 0.5")
abline(h = 0.8, col = "red", lty = 2)
text(60, 0.82, "Target Power = 0.8", col = "red")
```

Output :

POWER ANALYSIS AND SAMPLE SIZE CALCULATION

>>> Calculating required sample size per group (medium effect size, 80% power):

Two-sample t test power calculation

n = 63.76561

d = 0.5

sig.level = 0.05

power = 0.8

alternative = two.sided

NOTE: n is number in *each* group

>>> Calculating power with only 30 samples per group:

Two-sample t test power calculation

n = 30

d = 0.5

sig.level = 0.05

power = 0.4778965

alternative = two.sided

NOTE: n is number in *each* group

>>> Plotting Power vs. Sample Size (Effect size d = 0.5):

18.ADVANCED DATA HANDLING

Source code :

```
# Load tidyverse (includes dplyr and tidyr)
if (!require(tidyverse)) install.packages("tidyverse")
library(tidyverse)

cat("ORIGINAL DATA\n")

# Sample data: Student test scores
students <- data.frame(
  Name = c("Alice", "Bob", "Charlie", "David", "Eva", "Frank"),
  Gender = c("F", "M", "M", "M", "F", "M"),
  Score_Math = c(88, 75, 90, 85, 92, 60),
  Score_English = c(90, 80, 85, 88, 95, 70)
)
print(students)

# 1. Filter students with Math score >= 85
cat("\nFILTER: Math Score >= 85\n")
filtered_data <- students %>% filter(Score_Math >= 85)
print(filtered_data)

# 2. Select only Name and English Score
cat("\nSELECT: Name and English Score\n")
selected_data <- students %>% select(Name, Score_English)
print(selected_data)
```

3. Create new column: Average Score

```
cat("\nMUTATE: Add Average Score\n")
```

```
mutated_data <- students %>%
```

```
mutate(Avg_Score = (Score_Math + Score_English)/2)
```

```
print(mutated_data)
```

4. Group by Gender and summarize average scores

```
cat("\nGROUP BY Gender and summarize\n")
```

```
grouped_summary <- mutated_data %>%
```

```
group_by(Gender) %>%
```

```
summarise(
```

```
  Avg_Math = mean(Score_Math),
```

```
  Avg_English = mean(Score_English),
```

```
  Avg_Total = mean(Avg_Score)
```

```
)
```

```
print(grouped_summary)
```

5. Sort by Avg_Score descending

```
cat("\nARRANGE by Avg Score (descending)\n")
```

```
arranged_data <- mutated_data %>%
```

```
arrange(desc(Avg_Score))
```

```
print(arranged_data)
```

6. Pivot longer: convert subject scores into long format

```
cat("\nPIVOT LONGER: Convert scores to long format\n")
```

```
long_data <- students %>%
```

```
pivot_longer(  
  cols = starts_with("Score"),  
  names_to = "Subject",  
  values_to = "Score"  
)  
print(long_data)
```

Output :

ORIGINAL DATA

	Name	Gender	Score_Math	Score_English
1	Alice	F	88	90
2	Bob	M	75	80
3	Charlie	M	90	85
4	David	M	85	88
5	Eva	F	92	95
6	Frank	M	60	70

FILTER: Math Score $\geq$ 85

	Name	Gender	Score_Math	Score_English
1	Alice	F	88	90
2	Charlie	M	90	85
3	David	M	85	88
4	Eva	F	92	95

SELECT: Name and English Score

	Name	Score_English
1	Alice	90
2	Bob	80
3	Charlie	85
4	David	88
5	Eva	95
6	Frank	70

MUTATE: Add Average Score

	Name	Gender	Score_Math	Score_English	Avg_Score
1	Alice	F	88	90	89.0
2	Bob	M	75	80	77.5
3	Charlie	M	90	85	87.5
4	David	M	85	88	86.5
5	Eva	F	92	95	93.5
6	Frank	M	60	70	65.0

GROUP BY Gender and summarize

A tibble: 2 × 4

	Gender	Avg_Math	Avg_English	Avg_Total
	<chr>	<dbl>	<dbl>	<dbl>
1	F	90	92.5	91.2
2	M	77.5	80.8	79.1

ARRANGE by Avg Score (descending)

	Name	Gender	Score_Math	Score_English	Avg_Score
1	Eva	F	92	95	93.5
2	Alice	F	88	90	89.0
3	Charlie	M	90	85	87.5
4	David	M	85	88	86.5
5	Bob	M	75	80	77.5
6	Frank	M	60	70	65.0

PIVOT LONGER: Convert scores to long format

A tibble: 12 × 4

	Name	Gender	Subject	Score
	<chr>	<chr>	<chr>	<dbl>
1	Alice	F	Score_Math	88
2	Alice	F	Score_English	90
3	Bob	M	Score_Math	75
4	Bob	M	Score_English	80
5	Charlie	M	Score_Math	90
6	Charlie	M	Score_English	85
7	David	M	Score_Math	85
8	David	M	Score_English	88
9	Eva	F	Score_Math	92
10	Eva	F	Score_English	95
11	Frank	M	Score_Math	60
12	Frank	M	Score_English	70

19.MULTIPLE REGRESSION

Source code :

```
# Multiple Linear Regression in R

# Sample data: students' performance
students <- data.frame(
  Hours_Studied = c(2, 3, 5, 9, 10, 4, 6, 8, 9, 7),
  Classes_Attended = c(3, 4, 6, 7, 9, 10, 5, 6, 9, 9),
  Final_Score = c(50, 55, 65, 70, 85, 90, 60, 72, 88, 77)
)

# View data

# Fit multiple regression model
model <- lm(Final_Score ~ Hours_Studied + Classes_Attended, data =
students)

cat("Input Data:\n")
print(students)

# View model summary
cat("\nRegression Summary:\n")
print(summary(model))

# Predict scores for new data
new_data <- data.frame(
  Hours_Studied = c(6, 8),
  Classes_Attended = c(7, 9)
)
```

```
predicted <- predict(model, newdata = new_data)
```

```
cat("\nPredictions for New Students:\n")
```

```
print(cbind(new_data, Predicted_Score = predicted))
```


Output :

Input Data:

	Hours_	Studied	Classes_	Attended	Final_Score
1	2		3		50
2	3		4		55
3	5		6		65
4	9		7		70
5	10		9		85
6	4		10		90
7	6		5		60
8	8		6		72
9	9		9		88
10	7		9		77

Regression Summary:

Call:

```
lm(formula = Final_Score ~ Hours_Studied + Classes_Attended,  
    data = students)
```

Residuals:

Min	1Q	Median	3Q	Max
-6.2935	-1.4122	0.3394	2.0405	4.4718

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	32.2839	3.8403	8.407	6.63e-05 ***
Hours_Studied	0.3714	0.5557	0.668	0.525
Classes_Attended	5.3789	0.6384	8.425	6.54e-05 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.724 on 7 degrees of freedom

Multiple R-squared: 0.9441, Adjusted R-squared: 0.9282

F-statistic: 59.16 on 2 and 7 DF, p-value: 4.12e-05

Predictions for New Students:

	Hours_Studied	Classes_Attended	Predicted_Score
1	6	7	72.16436
2	8	9	83.66488

20.SIMPLE LINEAR MODEL

Source code :

```
# Sample dataset: height and weight of individuals
height <- c(150, 160, 165, 170, 175, 180, 185, 190)
weight <- c(50, 55, 58, 63, 65, 70, 72, 75)

# Create a data frame
data <- data.frame(Height = height, Weight = weight)

# Display the data
cat("INPUT DATA\n")
print(data)

# Fit a simple linear model
model <- lm(Weight ~ Height, data = data)

# Display summary of the model
cat("\nMODEL SUMMARY\n")
print(summary(model))

# Predict weight for new heights
new_height <- data.frame(Height = c(160, 175, 185))
predicted_weight <- predict(model, newdata = new_height)

# Show predictions
cat("\nPREDICTED WEIGHT\n")
```

```
print(cbind(new_height, Predicted_Weight = predicted_weight))
```

```
# Plot the data and the regression line
```

```
plot(data$Height, data$Weight,
```

```
      main = "Height vs. Weight",
```

```
      xlab = "Height (cm)",
```

```
      ylab = "Weight (kg)",
```

```
      pch = 16, col = "blue")
```

```
abline(model, col = "red", lwd = 2)
```

Output :

INPUT DATA

Height Weight

1	150	50
2	160	55
3	165	58
4	170	63
5	175	65
6	180	70
7	185	72
8	190	75

MODEL SUMMARY

Call:

```
lm(formula = Weight ~ Height, data = data)
```

Residuals:

Min	1Q	Median	3Q	Max
-1.0200	-0.5927	-0.1817	0.7299	1.2055

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-48.49875	4.25050	-11.41	2.72e-05 ***

Height 0.65163 0.02467 26.42 1.94e-07 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.871 on 6 degrees of freedom

Multiple R-squared: 0.9915, Adjusted R-squared: 0.9901

F-statistic: 698 on 1 and 6 DF, p-value: 1.941e-07

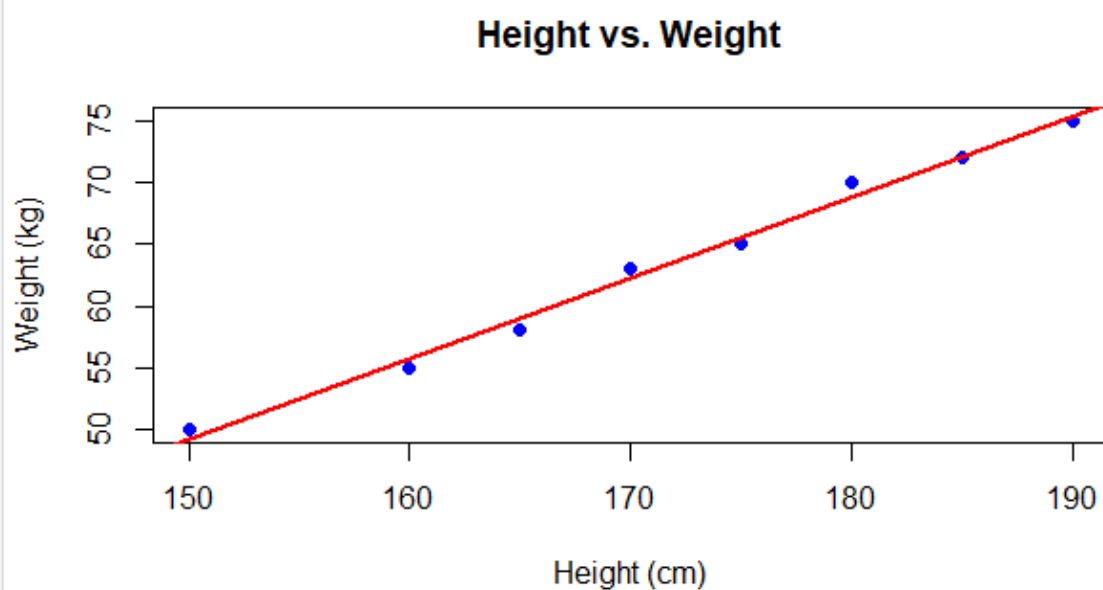
PREDICTED WEIGHT

Height Predicted_Weight

1 160 55.76190

2 175 65.53634

3 185 72.05263



21.LOGISTIC REGRESSION

Source code :

```
# Install and load brglm package if not installed
if (!require(brglm)) install.packages("brglm", dependencies = TRUE)
library(brglm)

# Sample dataset: student performance
students <- data.frame(
  Hours_Studied = c(1, 2, 2, 3, 4, 4, 5, 6, 7, 8),
  Attendance = c(2, 3, 2, 4, 5, 4, 6, 6, 7, 8),
  Passed = c(0, 0, 0, 0, 1, 1, 1, 1, 1, 1) # 0 = Fail, 1 = Pass
)

# View dataset
cat("INPUT DATA:\n")
print(students)

# Fit bias-reduced logistic regression model
log_model <- brglm(Passed ~ Hours_Studied + Attendance,
  data = students, family = binomial)

# View model summary
cat("\nLOGISTIC REGRESSION SUMMARY (Bias-Reduced):\n")
print(summary(log_model))

# Predict probabilities for new students
```

```
new_data <- data.frame(  
  Hours_Studied = c(3, 5, 7),  
  Attendance = c(4, 5, 7)  
)  
  
predicted_probs <- predict(log_model, newdata = new_data, type =  
"response")  
  
# Classify outcome (threshold = 0.5)  
predicted_class <- ifelse(predicted_probs > 0.5, 1, 0)  
  
# Show predictions  
cat("\nPREDICTIONS:\n")  
print(cbind(  
  new_data,  
  Predicted_Probability = round(predicted_probs, 3),  
  Predicted_Pass = predicted_class  
))
```


Output :

INPUT DATA:

	Hours_Studied	Attendance	Passed
1	1	2	0
2	2	3	0
3	2	2	0
4	3	4	0
5	4	5	1
6	4	4	1
7	5	6	1
8	6	6	1
9	7	7	1
10	8	8	1

LOGISTIC REGRESSION SUMMARY (Bias-Reduced):

Call:

```
brglm(formula = Passed ~ Hours_Studied + Attendance, family =  
binomial,  
      data = students)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-3.7960	3.0895	-1.229	0.219

Hours_Studied	1.6288	2.0906	0.779	0.436
Attendance	-0.4187	1.9708	-0.212	0.832

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 8.7741 on 9 degrees of freedom

Residual deviance: 3.5823 on 7 degrees of freedom

Penalized deviance: 4.29109

AIC: 9.5823

PREDICTIONS:

	Hours_Studied	Attendance	Predicted_Probability	Predicted_Pass
1	3	4	0.358	0
2	5	5	0.905	1
3	7	7	0.991	1

23.RATES AND POISSON REGRESSION

Source code :

```
# Sample data: accidents on roads

data <- data.frame(
  accidents = c(5, 10, 15, 25, 35),
  cars_per_day = c(1000, 2000, 3000, 4000, 5000),
  days_observed = c(30, 30, 30, 30, 30),
  road_type = factor(c("Urban", "Urban", "Highway", "Highway",
    "Highway"))
)

# Calculate exposure (total cars passed)
data$exposure <- data$cars_per_day * data$days_observed

cat("INPUT DATA:\n")
print(data)

# Fit Poisson regression model
model <- glm(accidents ~ road_type + offset(log(exposure)),
  family = poisson, data = data)

cat("\nPOISSON REGRESSION SUMMARY:\n")
print(summary(model))

# Predict expected number of accidents
pred <- predict(model, type = "response")
```

```
cat("\nPREDICTED ACCIDENT COUNTS:\n")  
print(cbind(data, Predicted_Accidents = round(pred, 2)))
```

Output :

INPUT DATA:

	accidents	cars_per_day	days_observed	road_type	exposure
1	5	1000	30	Urban	30000
2	10	2000	30	Urban	60000
3	15	3000	30	Highway	90000
4	25	4000	30	Highway	120000
5	35	5000	30	Highway	150000

POISSON REGRESSION SUMMARY:

Call:

```
glm(formula = accidents ~ road_type + offset(log(exposure)),  
     family = poisson, data = data)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-8.4764	0.1155	-73.408	<2e-16 ***
road_typeUrban	-0.2231	0.2828	-0.789	0.43

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

Null deviance: 1.8924 on 4 degrees of freedom

Residual deviance: 1.2387 on 3 degrees of freedom

AIC: 27.895

Number of Fisher Scoring iterations: 4

PREDICTED ACCIDENT COUNTS:

	accidents	cars_per_day	days_observed	road_type	exposure	Predicted_Accidents
1	5	1000	30	Urban	30000	5.00
2	10	2000	30	Urban	60000	10.00
3	15	3000	30	Highway	90000	18.75
4	25	4000	30	Highway	120000	25.00
5	35	5000	30	Highway	150000	31.25

24. NONLINEAR CURVE FITTING

Source code :

```
# Sample data: Exponential growth
x <- c(1, 2, 3, 4, 5, 6, 7, 8)
y <- c(2.3, 2.9, 4.2, 6.1, 9.0, 13.1, 19.0, 27.5)

# Plot raw data
plot(x, y, main = "Nonlinear Curve Fitting",
     xlab = "x", ylab = "y",
     pch = 16, col = "blue")

# Nonlinear model:  $y = a * \exp(b * x)$ 
nls_model <- nls(y ~ a * exp(b * x),
                start = list(a = 1, b = 0.2)) # initial guesses

# Show model summary
cat("NONLINEAR MODEL SUMMARY\n")
print(summary(nls_model))

# Predict fitted values
fitted_y <- predict(nls_model)

# Add fitted curve to the plot
lines(x, fitted_y, col = "red", lwd = 2)

# Compare actual vs predicted
```

```
cat("\nACTUAL VS PREDICTED\n")
print(data.frame(
  x = x,
  Actual = y,
  Predicted = round(fitted_y, 2)
))
```


Output :

NONLINEAR MODEL SUMMARY

Formula: $y \sim a * \exp(b * x)$

Parameters:

	Estimate	Std. Error	t value	Pr(> t)
a	1.400066	0.026821	52.2	3.32e-09 ***
b	0.372284	0.002652	140.4	8.80e-12 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.1254 on 6 degrees of freedom

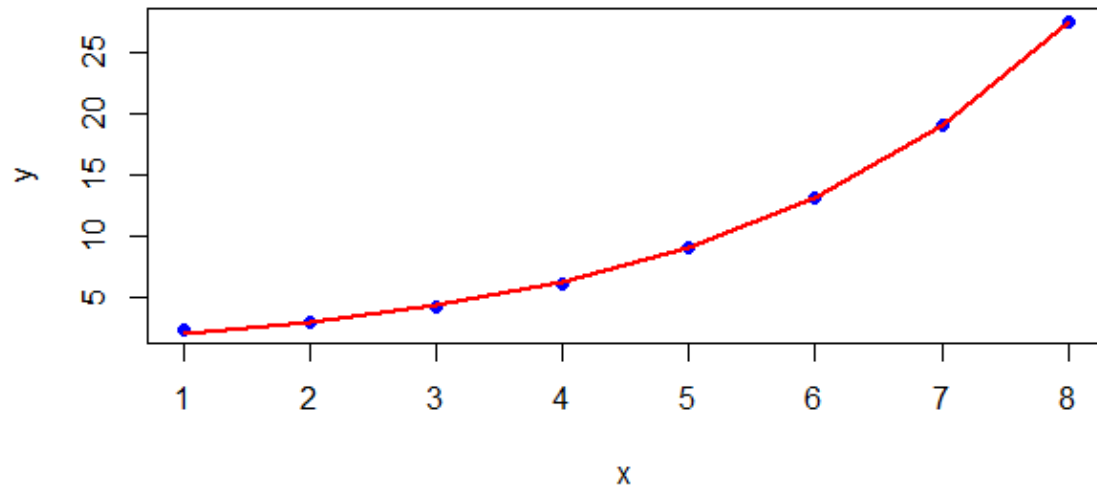
Number of iterations to convergence: 6

Achieved convergence tolerance: 4.198e-08

ACTUAL VS PREDICTED

x	Actual	Predicted
1	2.3	2.03
2	2.9	2.95
3	4.2	4.28
4	6.1	6.21
5	9.0	9.01
6	13.1	13.07
7	19.0	18.96
8	27.5	27.52

Nonlinear Curve Fitting



22.SURVIVAL ANALYSIS

Source code :

```
# Load required package
if (!require(survival)) install.packages("survival", dependencies = TRUE)
library(survival)
```

```
# Sample dataset: patients under treatment
```

```
data <- data.frame(
  time = c(5, 8, 10, 12, 15, 22, 30, 35, 45, 50), # time until event or
  censoring
  status = c(1, 1, 0, 1, 1, 0, 1, 1, 0, 1),      # 1 = event occurred, 0 =
  censored
  group = c("A", "A", "A", "A", "B", "B", "B", "B", "B", "B")
)
```

```
cat("INPUT DATA:\n")
```

```
print(data)
```

```
# Create a survival object
```

```
surv_obj <- Surv(time = data$time, event = data$status)
```

```
cat("\nSURVIVAL OBJECT:\n")
```

```
print(surv_obj)
```

```
# Fit Kaplan-Meier curve
```

```
km_fit <- survfit(surv_obj ~ group, data = data)
```

```
cat("\nKAPLAN-MEIER SUMMARY:\n")
```

```
print(summary(km_fit))
```

```
# Plot survival curves
```

```
plot(km_fit, col = c("blue", "red"), lwd = 2,
```

```
     xlab = "Time", ylab = "Survival Probability",
```

```
     main = "Kaplan-Meier Survival Curves")
```

```
legend("bottomleft", legend = levels(factor(data$group)),
```

```
     col = c("blue", "red"), lwd = 2)
```

```
# Perform Log-Rank Test to compare groups
```

```
cat("\nLOG-RANK TEST:\n")
```

```
log_rank <- survdiff(surv_obj ~ group, data = data)
```

```
print(log_rank)
```

Output:

```
time status group
1    5     1    A
2    8     1    A
3   10     0    A
4   12     1    A
5   15     1    B
6   22     0    B
7   30     1    B
8   35     1    B
9   45     0    B
10  50     1    B
```

SURVIVAL OBJECT:

```
[1] 5  8 10+ 12 15 22+ 30 35 45+ 50
```

KAPLAN-MEIER SUMMARY:

Call: `survfit(formula = surv_obj ~ group, data = data)`

group=A

time	n.risk	n.event	survival	std.err	lower 95% CI	upper 95% CI
------	--------	---------	----------	---------	--------------	--------------

5	4	1	0.75	0.217	0.426	1
8	3	1	0.50	0.250	0.188	1
12	1	1	0.00	NaN	NA	NA

group=B

time	n.risk	n.event	survival	std.err	lower 95% CI	upper 95% CI
------	--------	---------	----------	---------	--------------	--------------

15	6	1	0.833	0.152	0.583	1
30	4	1	0.625	0.213	0.320	1
35	3	1	0.417	0.222	0.147	1

50	1	1	0.000	NaN	NA	NA
----	---	---	-------	-----	----	----

LOG-RANK TEST:

Call:

```
survdif(formula = surv_obj ~ group, data = data)
```

	N	Observed	Expected	(O-E)^2/E	(O-E)^2/V
group=A	4	3	0.876	5.148	7.71
group=B	6	4	6.124	0.737	7.71

Chisq= 7.7 on 1 degrees of freedom, p= 0.005

```
> source("~/active-rstudio-document")
```

INPUT DATA:

	time	status	group
1	5	1	A
2	8	1	A
3	10	0	A
4	12	1	A
5	15	1	B
6	22	0	B
7	30	1	B
8	35	1	B
9	45	0	B
10	50	1	B

SURVIVAL OBJECT:

```
[1] 5 8 10+ 12 15 22+ 30 35 45+ 50
```

KAPLAN-MEIER SUMMARY:

```
Call: survfit(formula = surv_obj ~ group, data = data)
```

group=A

time	n.risk	n.event	survival	std.err	lower 95% CI	upper 95% CI
5	4	1	0.75	0.217	0.426	1
8	3	1	0.50	0.250	0.188	1
12	1	1	0.00	NaN	NA	NA

group=B

time	n.risk	n.event	survival	std.err	lower 95% CI	upper 95% CI
15	6	1	0.833	0.152	0.583	1
30	4	1	0.625	0.213	0.320	1
35	3	1	0.417	0.222	0.147	1
50	1	1	0.000	NaN	NA	NA

LOG-RANK TEST:

Call:

survdifff(formula = surv_obj ~ group, data = data)

N observed Expected (O-E)^2/E (O-E)^2/V

group=A 4 3 0.876 5.148 7.71

group=B 6 4 6.124 0.737 7.71

Chisq= 7.7 on 1 degrees of freedom, p= 0.005

Kaplan-Meier Survival Curves

